Purr Engine

Éditeur de scène 3D temps réel

Cours	IMN401 (Hiver 2026)
Projet	Purr - éditeur / scène 3D
Équipe	Individuelle
Étudiante	Zoe Stoll
GitHub	zoexshl/Purr

3.1 Sommaire

Purr est une application C++/OpenGL qui permet de construire et d'éditer des scènes 3D en temps réel

Le projet est organisé autour de deux composantes :

- **Le moteur Purr :** fenêtre, boucle d'application, événements, rendu, shaders, buffers, caméra, textures et interface ImGui.
- **L'application Sandbox :** éditeur de scène avec hiérarchie, sélection, gizmos, import de modèles, animation, lumières, etc.

Objectifs principaux du projet :

- Offrir un environnement d'édition de scène interactif complet.
- Démontrer un maximum de critères fonctionnels du cours (géométrie, transformation, caméra, texture, illumination).
- Produire une application compilable et reproductible sur Windows, avec toutes les dépendances intégrées (vendor submodules).

L'application démarre avec une scène vide. L'utilisateur peut ajouter des primitives, importer des modèles OBJ/FBX, configurer des matériaux et lumières, animer des objets, naviguer avec une caméra orbitale et switcher entre les modes d'édition et de lecture.

3.2 Intéractivité

Tout tourne autour d'une boucle d'événements temps réel avec une interface ImGui entièrement customisée.

Entrées

- Clavier (WASD + flèches) et souris pour la navigation caméra.
- Clic sur les objets dans la hiérarchie ou dans la vue pour les sélectionner.
- Ctrl+Clic pour la sélection multiple.
- Raccourcis T / R / S pour les modes translation / rotation / échelle.
- Ctrl+Z / Ctrl+Y pour undo/redo. Del pour supprimer. Ctrl+C / Ctrl+V pour copier-coller.
- Barre espace pour Play/Stop. Ctrl+S / Ctrl+O pour sauvegarde/chargement de scène.
- Import de modèles via boîte de dialogue (OBJ ou FBX).

Sorties

- Rendu 3D temps réel dans un framebuffer affiché dans la vue principale.
- Modes multi-vue : Single, Dual, Quad (Perspective + Top + Front + Right simultanément).
- Panneaux UI : hiérarchie d'objets, propriétés, lumières, keyframes, timeline, logs.
- Rétroaction visuelle : gizmos colorés, boîtes de délimitation, icônes de type d'objet, curseur dynamique.

Formes d'interactivité notable

- Hiérarchie avec parentage drag-and-drop, renommage inline, propagation des transformations parent → enfant.
- Gizmos 3D avec axes X/Y/Z indépendants & scaling uniforme via un cube central.
- Caméra orbitale en édition + caméra TPS/FPS en mode Play.
- Système d'animation : keyframes éditables dans un panneau dédié, lecture/pause/stop/loop.
- Animation physique par masses-ressorts (spring) configurable par objet.
- Mode Play avec snapshot/restore de la scène complète.
- Curseur dynamique qui change selon l'outil actif (main, flèches, etc.).

3.3 Technologie

Langages et bibliothèques

Technologie	Utilisation
C++17	Moteur et application principale

Technologie	Utilisation
OpenGL	Pipeline de rendu 3D
GLSL	Shaders vertex/fragment
GLFW	Fenêtre native et gestion des inputs
Glad	Chargement des fonctions OpenGL
ImGui (docking)	Interface utilisateur en mode immédiat
GLM	Mathématiques : vecteurs, matrices, transformations
Assimp	Import de modèles 3D (FBX, OBJ, etc.)
stb_image	Chargement de textures (PNG, JPG, etc.)
spdlog	Journalisation interne
Premake5	Génération du projet Visual Studio

Outils de développement

- Visual Studio 2022, toolset MSVC v143, configuration x64.
- Profils de build : Debug, Release, Dist.
- Git avec submodules pour les dépendances (GLFW, ImGui, spdlog, Glad, Assimp).
- MIXANO : animation

Justification des choix

OpenGL + C++ donne un contrôle complet du pipeline graphique, ce qui était important pour implémenter les shaders GLSL custom (illumination, procédural, UV tiling). ImGui accélère énormément la construction d'outils d'édition — tout le panneau de propriétés, la hiérarchie et les gizmos s'appuient dessus. Assimp simplifie l'import multi-format, même si ça reste compliqué en pratique (voir section 3.8).

3.4 Compilation

Prérequis

- Windows 10/11 x64
- Visual Studio 2022 (C++ Desktop Development)
- Premake5 (exécutable dans le PATH ou à la racine)

Procédure recommandée

1. Cloner le dépôt avec les submodules :
   `git clone --recurse-submodules https://github.com/zoexshl/Purr.git`
2. Ouvrir une console à la racine et lancer :
   `GenerateProject.bat`
3. Ouvrir la solution .sln générée dans Visual Studio 2022.
4. Sélectionner la configuration Debug (ou Release) en x64.
5. Compiler la solution (Ctrl+Shift+B) et lancer Sandbox.

Dépendances et DLL

Toutes les bibliothèques tierces sont dans Purr/vendor (submodules Git). Le script premake5.lua configure les liens et copie automatiquement les DLL Assimp dans le dossier de sortie de Sandbox via une règle post-build.

Vérification minimale

- L'application démarre sans crash.
- La fenêtre de rendu apparaît avec la scène par défaut.
- On peut ajouter un cube et le sélectionner dans la hiérarchie.

3.5 Architecture

L'architecture suit une séparation moteur / application, inspirée du Game Engine Series de The Cherno (Hazel Engine), adaptée pour un éditeur 3D avec uniquement le backend OpenGL.

Couche moteur — Purr

- Application/loop : Application.cpp, boucle principale, gestion de la LayerStack.
- Fenêtre/événements : WindowsWindow, Input, système d'événements (KeyEvent, MouseEvent, ApplicationEvent).
- Rendu : RenderCommand, VertexArray, Buffer, Shader, Texture, Framebuffer, Camera.
- Assets : ObjLoader, AssimpLoader, TextureManager.
- UI : ImGuiLayer (ImGui docking + GLFW/OpenGL3 backends modernes).

Couche application — Sandbox

- ExampleLayer : couche principale qui contient toute la logique de l'éditeur.
- SceneObject : structure de données d'un objet de scène (mesh, matériau, transform, animation, script...).
- Outils d'édition : hiérarchie, sélection multiple, gizmos, import, undo/redo.
- Systèmes : animation classique (keyframes TRS), animation physique (spring), coordonnées polaires.

Flux d'exécution

6. Initialisation fenêtre + contexte OpenGL (GLFW + Glad).
7. Initialisation ressources de rendu : shaders, framebuffers, meshes, textures.
8. Boucle principale : OnUpdate (logique) → rendu de scène → rendu UI ImGui.
9. Traitement événements (clavier, souris, resize, fermeture).

3.6 Fonctionnalités (critères fonctionnels)

#	Catégorie	Critère	Preuve technique
1	**Image**	Importation d'images	*OpenFileDialog() + TextureManager::Load()*
2	**Image**	Histogramme (canal R/G/B)	*ComputeHistogram() + affichage dans panneau Scene*
3	**Transformation**	Graphe de scène	*ParentIndex, GetWorldTransform(), DrawHierarchyNode(), drag-and-drop*
4	**Transformation**	Sélection multiple	*m_Selection (unordered_set), Ctrl+Click, opérations bulk*
5	**Transformation**	Transformations interactives	*Gizmos T/R/S, axes XYZ, raccourcis clavier*
6	**Transformation**	Undo / Redo	*m_UndoStack, m_RedoStack, SaveSnapshot(), Ctrl+Z / Ctrl+Y*
7	**Transformation**	Coordonnées non cartésiennes	*PolarMarker : cylindrique (r, θ, Y), PolarSyncCartesianFromPolar()*
8	**Transformation**	Animation classique (keyframes)	*KeyframeTRS, SampleKeys(), interpolation linéaire, play/pause/stop/loop*
9	**Transformation**	Animation physique (spring)	*SpringK, SpringDamping, SpringMass, UpdateSpringAnimations()*
10	**Géométrie**	Boîte de délimitation	*m_BBoxVA, BuildBBoxMesh(), wireframe sur la sélection*
11	**Géométrie**	Primitives géométriques (≥2)	*Cube, Sphère UV, Triangle, Cercle, Polygone, Ellipse — 6 primitives*
12	**Géométrie**	Modèles 3D (multi-format)	*ObjLoader (OBJ), AssimpLoader (FBX/autres), ImportFBXGrouped()*
13	**Caméra**	Point de vue / navigation	*Orbite + zoom + pan; WASD éditeur; TPS/FPS en Play*
14	**Caméra**	Modes de projection	*ProjectionMode::Perspective et ::Orthographic*
15	**Caméra**	Multi-vue (agencement)	*Single/Dual/Quad, 4 framebuffers : Perspective, Top, Front, Right*
16	**Texture**	Coordonnées de texture (UV)	*UV sphere, OBJ avec UVs, u_TilingFactor par objet*
17	**Texture**	Texture procédurale	*Damier GLSL (fract(sin(dot(...))), échelle ajustable*
18	**Illumination**	Modèles d'illumination (≥3)	*Lambert, Phong, Blinn-Phong — enum IlluminationModel*
19	**Illumination**	Matériaux (≥3 propriétés)	*Ambient, Diffuse, Specular, Shininess par objet*
20	**Illumination**	Lumières multiples (≥4)	*m_Lights[4], position/couleur/intensité/atténuation, panneau Lumières*
21	**Image**	Sauvegarder/charger une scène	*Sérialisation JSON, sauvegarde keyframes et état complet*
22	**Transformation**	Curseur dynamique	*ImGui::SetMouseCursor() — 5+ états selon gizmo actif*

#	Catégorie	Critère	Preuve technique
23	**Illumination**	Types de lumière distincts	*4 instances ponctuelles + ambiante globale; types directionnel/spot non implémentés*

Détail des critères principaux

1. Import de modèles 3D

Deux chargeurs sont disponibles :

1 - **ObjLoader.cpp** pour les fichiers OBJ (avec gestion des UV et textures via m_ObjTexCache)

2- **AssimpLoader.cpp** pour les FBX.

L'import FBX gère le multi-maillage avec **ImportFBXGrouped()** qui regroupe les maillages sous un objet parent.

2. Animation classique par keyframes

Chaque objet peut avoir une liste de **KeyframeTRS** (position, rotation, échelle, temps). La fonction **SampleKeys**() interpole linéairement entre les keyframes au temps courant.

L'interface permet d'ajouter, modifier et supprimer des keyframes depuis le panneau ***Properties***.

La lecture ***(Play/Pause/Stop)*** est indépendante par objet et supporte la boucle.

3. Animation physique (spring)

Un objet peut avoir un composant ressort défini par SpringK (raideur), SpringDamping (amortissement) et SpringMass (masse).

La simulation est mise à jour chaque frame dans UpdateSpringAnimations() avec intégration de Verlet. L'objet oscille autour de sa position de repos.

4. Graphe de scène et parentage

Chaque SceneObject possède un **ParentIndex**.

La transformation monde est calculée récursivement via **GetWorldTransform**(). Le drag-and-drop dans la hiérarchie ImGui permet de reparenter des objets avec préservation de la transformation monde (décomposition GLM via *glm::decompose*).

5. Système de coordonnées polaires (cylindrique)

Un type spécial **PolarMarker** permet de positionner des objets avec des coordonnées cylindriques (r, θ, Y).

La synchronisation avec les coordonnées cartésiennes se fait via PolarSyncCartesianFromPolar(). C'est une façon originale de placer des objets sur un cercle ou une trajectoire circulaire!

6. Multi-vue (Quad layout)

En mode Quad, **4 framebuffers** distincts sont rendus simultanément (Perspective, Top, Front, Right) et affichés dans la même fenêtre ImGui.

Chaque vue a sa propre caméra avec orientation fixe (orthographique pour Top/Front/Right, perspective pour la vue principale).

3.7 Ressources

Ressources produites par l'équipe

- ✓ Tous les shaders GLSL (vertex/fragment) — illumination Lambert/Phong/Blinn-Phong, texture procédurale, wireframe, gizmos.
- ✓ Le moteur Purr complet (couche Application, Rendering, ImGuiLayer, Camera, Framebuffer, Buffers).
- ✓ L'éditeur Sandbox avec toute la logique de scène (SandboxApp.cpp).
- ✓ ObjLoader.cpp : chargeur OBJ custom avec gestion des UV et cache de textures.
- ✓ TextureManager.cpp : gestionnaire de textures avec cache.

Ressources externes

- **Bibliothèques:** GLFW, Glad, ImGui (branche docking), GLM, Assimp, stb_image, spdlog.
- **Référence principale:** The Cherno — Game Engine Series (YouTube/GitHub Hazel Engine).
- **Modèles 3D de test :** formats OBJ et FBX trouvés en ligne (Sketchfab, exportés depuis Roblox Studio, etc.).
- **Textures de test :** fichiers PNG/JPG libres de droits.

IA et divulgation

L'IA (Claude / ChatGPT) a été utilisée comme outil d'assistance au développement : débogage de code C++/GLSL, aide à la rédaction de ce document, et suggestions d'implémentation pour certains systèmes (gizmos, serialisation JSON, spring physics). Tout le code final a été relu, compris et intégré manuellement.

3.8 Présentation de l' Équipe

Membres

Prénom	Nom	Rôle
Zoe	Stoll	Développement complet (équipe individuelle)

Répartition des tâches (solo)

- Moteur et rendu : architecture du moteur Purr, shaders, pipeline OpenGL, framebuffers, caméra.
- Interface et outils d'édition : ImGui docking, hiérarchie, gizmos, sélection, undo/redo.
- Import d'assets et pipeline de données : ObjLoader, AssimpLoader, TextureManager, JSON scene.
- Fonctionnalités avancées : animations, spring physics, coordonnées polaires, multi-vue.
- Tests, stabilisation, vidéo et livraison.

Bilan — Défis techniques rencontrés

Import de modèles 3D (OBJ/FBX) (défi principal)

C'est clairement le plus gros challenge du projet. Pour les OBJ, le problème principal était les textures : il fallait un cache parallèle (m_ObjTexCache) pour ne pas perdre les paths de texture sur les caches hits. L'orientation des UV était aussi un mystère au début : les exports Roblox Studio n'ont pas besoin de flip V, contrairement à ce qu'on voit partout!

Pour les FBX avec Assimp, c'était encore plus compliqué : les textures sont dans des sous-dossiers (source, texture) souvent avec des chemins absolus qui ne fonctionnent pas.

Il a fallu alors implémenter une *résolution de chemins relatifs* et un *fallback*.

Les modèles avec plusieurs maillages nécessitaient ImportFBXGrouped() pour créer un objet parent + des enfants.

Maps et collision (une limitation connue!)

L'import de maps (niveaux) a été partiellement réussi. Le vrai problème c'est qu'il n'y a pas de détection automatique des types de surfaces (mur, plancher, obstacle). En pratique, les maps sont importées comme meshes visuels et les collisions sont simulées avec des cubes invisibles placés manuellement.

Par conséquence, les maps ne sont pas parfaites ! On peut tomber de certaines zones si les cubes invisibles ne couvrent pas tout le sol, et les murs peuvent être traversés. C'est une limitation réelle et connue, qui nécessiterait un système de BVH ou de heightmap pour être réglée proprement.

Animation (keyframes + hiérarchies)

Le système de keyframes fonctionne bien pour les objets simples. La difficulté est venue avec les hiérarchies : animer un parent tout en laissant les enfants avec leurs propres animations demandait de bien gérer l'ordre d'évaluation dans UpdateClassicAnimations().

La génération de démo automatique ("Générer demo sur sélection") aide à tester rapidement.

Ce qui serait amélioré avec plus de temps

- Implémentation d'un vrai système de collision (BVH tree ou heightmap) pour les maps.

- Support complet des textures embarquées dans les FBX (actuellement dépend de chemins relatifs).
- Ajout d'une lumière directionnelle et d'un spot pour couvrir les 4 types de lumière distincts.
- Interface de timeline plus complète (scrubber, courbes d'interpolation).
- Export d'images/screenshots depuis le framebuffer (glReadPixels + stb_image_write).